

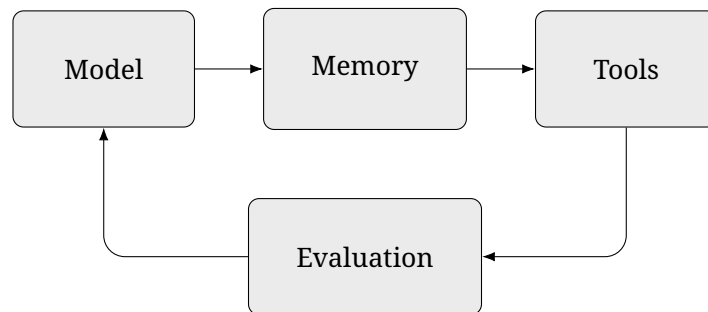
Enhancement Methods for Running Agentic AI Systems

A technical article on orchestration, memory, tools,
evaluation, and reliable autonomous workflows

Author: Saed Abdalgani

Date: June 11, 2026

Language: English, with a BiDi Hebrew technical demonstration



Contents

1	Introduction	1
2	Background and Theoretical Foundations	1
3	Enhancement Taxonomy	3
4	Reasoning and Planning Control	5
5	Retrieval, Memory, and Context Engineering	6
6	Tool Use, Sandboxing, and Action Reliability	7
7	Multi-Agent Orchestration	7
8	Evaluation, Observability, and Governance	8
9	Case Study: Autonomous Academic PDF Pipeline	9
10	Technical and Mathematical Reliability Section	10
11	Visual Analysis	11
12	Discussion	13
13	Limitations	13
14	Conclusion	14
	Bibliography	16

List of Figures

1	A minimal control loop for enhanced agentic AI. The model is only one component; reliability comes from the orchestration of state, tools, memory, evaluation, and human approval. . .	2
2	Raster image generated through the asset pipeline. The stack emphasizes that enhancements can target different layers and that feedback loops connect all layers.	3
3	Python-generated graph created with Matplotlib and saved as a PDF asset. It visualizes the expected tradeoff between incremental quality gains and relative run cost.	12

List of Tables

1	Major enhancement families for running agentic AI systems.	4
---	--	---

1 Introduction

Agentic artificial intelligence is the engineering practice of giving a language model a bounded ability to plan, call tools, observe results, revise its state, and continue toward a goal. A conventional chat system produces an answer after one prompt. An agentic system instead runs a controlled loop: it interprets a task, selects an action, uses a tool or another model call, stores observations, and decides whether the task is complete. This shift is attractive because many useful tasks are not single-turn text generation tasks. They involve reading files, querying databases, compiling code, scheduling work, checking outputs, and reacting to errors. The same shift is risky because every additional action increases cost, latency, failure surface, and the probability that a mistaken intermediate step becomes a visible final mistake.

This article studies enhancement methods for running agentic AI systems in a disciplined way. The term *enhancement* is used deliberately. The goal is not to make an agent merely more autonomous; it is to make it more reliable under explicit constraints. Some enhancements improve reasoning quality, such as plan-and-solve prompting and search over multiple candidate solution paths. Some improve grounding, such as retrieval-augmented generation and memory. Some improve execution, such as tool schemas, sandboxing, and idempotent actions. Others improve socio-technical control, such as human approval gates, trace logging, and evaluation suites. A system becomes production-worthy only when these enhancements are composed into a coherent architecture rather than bolted on as isolated tricks.

The modern research line behind agents combines several earlier ideas. Chain-of-thought prompting showed that intermediate reasoning can improve some multi-step tasks (Wei et al. 2022). ReAct connected reasoning traces with actions in an external environment (Yao, Zhao, et al. 2022). Toolformer demonstrated that language models can learn when and how to invoke simple external tools (Schick et al. 2023). Later work added reflection, tree search, multi-agent conversation, and embodied skill libraries (Shinn et al. 2023; Yao, Yu, et al. 2023; Wu et al. 2023; G. Wang et al. 2023). Production guidance has also matured: practical agent builders increasingly distinguish fixed workflows from open-ended agents, and reserve autonomy for cases where feedback can be observed and checked (Anthropic 2024).

The central thesis is that an effective agentic AI system is best viewed as a reliability budget. Every component consumes resources and reduces or increases uncertainty. A planner can reduce missing-step errors, but it introduces additional model calls. A retrieval system can reduce hallucination, but it introduces ranking, freshness, and chunking errors. A tool can produce exact answers, but it can also trigger destructive actions if permission boundaries are weak. An evaluator can catch mistakes, but only if its test oracle is meaningful. The engineering problem is therefore not simply “which framework should I use?” but “which set of enhancements moves the most important failure modes below an acceptable threshold for this task?”

The scope of the article is practical and architectural. It does not present a new benchmark or a new model architecture. Instead, it organizes established methods into a design framework for students, developers, and technical authors who need to build or evaluate systems based on CrewAI, LangGraph, AutoGen, the OpenAI Agents SDK, or similar orchestration layers (CrewAI 2026; LangChain 2026; OpenAI 2026). The running example is an autonomous academic PDF generation pipeline: an agent crew researches a topic, writes Markdown, creates figures, assembles LaTeX, compiles with LuaLaTeX and Biber, and performs quality assurance. This example is narrow enough to be concrete and broad enough to expose the core problems of tool use, artifacts, citations, multilingual text, and pass-based verification.

The article is organized as follows. Section 2 defines the theoretical foundations of agentic systems and introduces a vector diagram of the control loop. Section 3 classifies enhancement methods and presents a professional comparison table. Section 4 studies reasoning and planning controls. Section 5 covers retrieval, memory, and context engineering. Section 6 studies tool use and action reliability. Section 7 examines multi-agent orchestration. Section 8 explains evaluation and observability. Sections 9, 10, and 11 provide a case study, a mathematical reliability model, and a visual analysis. The final sections discuss tradeoffs, limitations, and conclusions.

2 Background and Theoretical Foundations

An agentic AI system can be defined as a computational arrangement in which a model is embedded inside a control loop that can observe state, choose actions, and update its behavior based on feedback.

This definition is broader than a chatbot and narrower than a fully autonomous robot. The distinguishing feature is not consciousness, intent, or open-ended agency. It is the operational connection between language-model inference and external state. A system that writes a SQL query, executes it, reads the result, and revises the query is already agentic in this engineering sense. A system that drafts an email but cannot inspect whether it was sent is less agentic because it lacks an observation-action loop.

The foundational abstraction is the *augmented language model*. A base model is augmented with instructions, context, tools, retrieval, memory, and output contracts. Reasoning methods then organize the model calls. Chain-of-thought encourages the model to externalize intermediate steps, but it does not itself guarantee that those steps are correct. ReAct adds actions and observations, turning a reasoning sequence into an environment-coupled loop (Yao, Zhao, et al. 2022). Plan-and-solve methods first ask for a decomposition and then execute the subtasks (L. Wang et al. 2023). Tree-of-thoughts methods represent multiple possible reasoning paths and search among them, which is useful when early choices strongly affect final outcomes (Yao, Yu, et al. 2023). These methods matter because many agent failures are not linguistic failures; they are control failures: omitted steps, premature stopping, stale context, and unverified assumptions.

Figure 1 presents the basic control loop used throughout the article. The user goal enters an orchestrator. The orchestrator routes work to a model, a memory or retrieval layer, tool adapters, and an evaluator. Observations return to the state store, and the loop continues until a termination condition is met. The figure is intentionally simple. It does not assume a particular framework. The same pattern can be implemented as a CrewAI crew, a LangGraph state machine, an AutoGen conversation, or a custom Python workflow. What changes is the degree of determinism, traceability, and developer control.

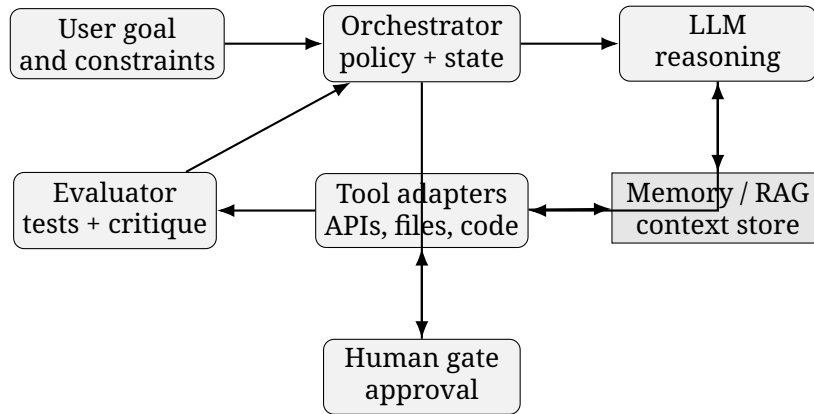


Figure 1: A minimal control loop for enhanced agentic AI. The model is only one component; reliability comes from the orchestration of state, tools, memory, evaluation, and human approval.

The loop in Figure 1 also clarifies why naive autonomy is insufficient. If the orchestrator has no explicit state, the system forgets what has been tried. If tool adapters expose raw APIs without validation, the agent can call them with malformed or unsafe arguments. If the evaluator only asks the same model whether the output looks good, the system may develop a false sense of certainty. If the human approval gate is placed too late, the user sees a polished but wrong artifact. Enhancement is therefore the placement of friction at the right points. Good friction slows down dangerous or uncertain actions and accelerates routine actions whose outputs are easy to verify.

A useful theoretical distinction is between *workflows* and *agents*. A workflow follows a mostly predefined sequence: retrieve documents, draft answer, run validator, return output. An agent has more freedom to choose steps, such as deciding whether to search, compute, ask a sub-agent, or stop. The distinction is not binary. Many real systems use a workflow spine with agentic pockets. For example, a PDF-generation pipeline may have fixed stages for source generation and compilation, while allowing an agent to decide how to repair a LaTeX error. The enhancement strategy should match the degree of freedom. More freedom requires better traces, stronger permissioning, and more explicit stopping rules.

Finally, agentic systems require a shift in evaluation. A single final answer is not enough. The developer must inspect trajectories: plans, observations, tool inputs, tool outputs, retries, and rejected alternatives. A system may reach a correct answer through a fragile path, or it may fail gracefully in a way that is

acceptable for a low-risk task. Reliability is therefore temporal. It is distributed across the sequence of actions, not concentrated in the last paragraph. This is why logs, compile journals, and QA reports are not bureaucratic artifacts; they are part of the agent’s epistemic surface. They make it possible to debug, reproduce, and improve the system.

3 Enhancement Taxonomy

Enhancement methods can be organized by the failure mode they address. This taxonomy is more useful than organizing by framework name because the same method often appears under different terminology in different platforms. A “crew”, a “graph”, a “swarm”, and a “router” can all implement similar control patterns. The practical question is which layer of the agent stack must be improved. Figure 2 shows the layered stack used in this article: model, action, context, orchestration, evaluation, and governance. The layers are not independent; each layer constrains the others. A high-quality retrieval layer is wasted if the planner never asks for evidence, and a strong evaluator is ineffective if tool traces are not preserved.

Layered Enhancement Stack for Agentic AI

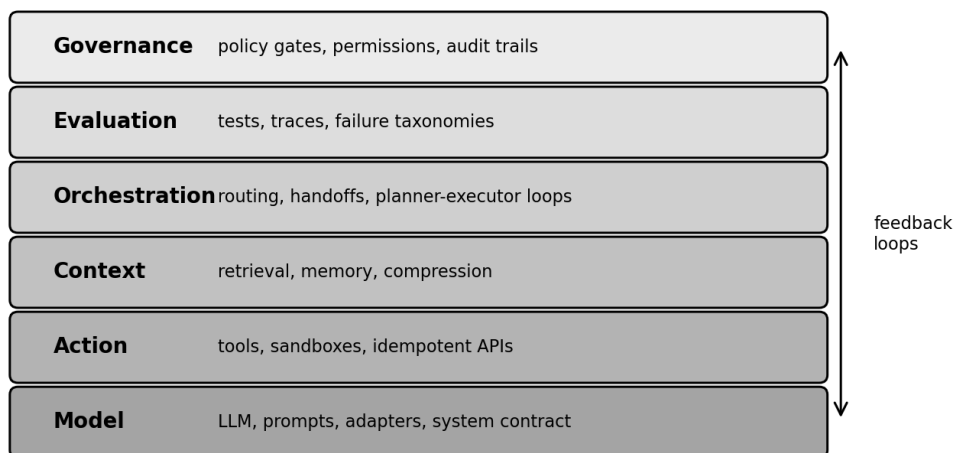


Figure 2: Raster image generated through the asset pipeline. The stack emphasizes that enhancements can target different layers and that feedback loops connect all layers.

The first class of enhancements controls reasoning. These methods include structured prompts, explicit plans, decomposition, candidate generation, self-consistency, tree search, and reflection. Their purpose is to reduce missing steps, overconfident shortcuts, and premature finalization. Plan-and-solve prompting is an example: the model is instructed to first devise a plan and then solve the subtasks in order (L. Wang et al. 2023). Tree-of-thoughts is a more expensive example: it explores multiple intermediate possibilities and uses evaluation to choose among them (Yao, Yu, et al. 2023). Reflection and self-refinement introduce feedback after an attempt, allowing the model to revise based on critique or task signals (Shinn et al. 2023; Madaan et al. 2023).

The second class controls grounding. Retrieval-augmented generation supplies evidence from a corpus at run time (Lewis et al. 2020). Long-term memory stores facts, preferences, and prior trajectories, while short-term working memory stores the current task state. Context compression summarizes or selects what should remain in the prompt when token budgets are limited. Grounding enhancements are crucial when the task depends on external documents, current files, or precise specifications. They also introduce their own hazards: irrelevant retrieval, stale chunks, duplicated memories, and summaries that erase exceptions.

The third class controls action. Tool use turns language into effects. The model may call a calculator, search tool, database, browser, compiler, calendar, email system, or file writer. Toolformer framed tool

use as a learnable decision about which API to call and how to incorporate the result (Schick et al. 2023). In production systems, tool use must be mediated by schemas, permissions, dry-run modes, timeouts, and audit logs. The tool layer should make illegal actions hard, ambiguous actions visible, and routine safe actions cheap.

The fourth class controls orchestration. Orchestration determines who does what, in which order, and under which stopping rule. A single-agent loop is simple but can become opaque. A multi-agent design can separate roles, such as researcher, writer, compiler, and reviewer, but it can also create coordination overhead. Conversation frameworks such as AutoGen focus on multi-agent exchanges (Wu et al. 2023). CrewAI emphasizes crews, tasks, tools, memory, and observability (CrewAI 2026). LangGraph emphasizes graph-based durable execution and human-in-the-loop control (LangChain 2026). The right topology depends on task decomposability, failure cost, and the need for reproducibility.

The fifth class controls evaluation and governance. Evaluation includes unit tests, benchmark tasks, simulation, judge models, trace review, and output-specific validators. Governance includes rate limits, permission scopes, approval gates, redaction, and incident response. These methods do not necessarily make the model smarter. Instead, they make the system safer and more diagnosable. For agentic systems, this distinction matters: the most valuable improvement may be a simple validator that prevents broken outputs from leaving the pipeline.

Table 1 summarizes the taxonomy. The table is intentionally operational: it maps each enhancement family to a common failure mode, a representative method, a primary benefit, and a main risk. The risk column is important because enhancements are not free. Many failures in agent systems are second-order failures introduced by an enhancement that was applied without a clear need.

Table 1: Major enhancement families for running agentic AI systems.

Enhancement family	Typical failure addressed	Representative method	Main benefit and main risk
Reasoning control	Missing steps, shallow plans, premature answers	Plan-and-solve, ReAct, tree search, reflection	Better decomposition and revision; risk of extra latency and self-justifying critique
Context and memory	Hallucination, stale assumptions, lost task state	RAG, episodic memory, context compression	Better grounding and continuity; risk of retrieval noise and memory drift
Tool reliability	Arithmetic errors, unverifiable claims, manual execution gaps	Typed tools, sandboxes, idempotent APIs, dry runs	Exact operations and reproducible actions; risk of unsafe side effects
Orchestration	Role confusion, unbounded loops, poor handoffs	Graph workflows, crews, routers, planner-executor loops	Clear division of labor; risk of coordination overhead
Evaluation and QA	Silent failure, unresolved references, brittle outputs	Unit tests, validators, judge rubrics, trace review	Detects faults before release; risk of false confidence when tests are weak
Governance	Excess autonomy, privacy leakage, policy violations	Permissions, human gates, audit logs, redaction	Safer operation and accountability; risk of slowing harmless tasks

A mature design rarely applies every method at maximum strength. Instead, the developer identifies bottlenecks. If the task is a deterministic transformation, a workflow with validators may be enough. If the task requires exploration, such as debugging an unfamiliar codebase, a planning and reflection layer may be valuable. If the task touches private data or external systems, governance must be designed before autonomy is increased. The key design habit is to describe each enhancement in terms of a failure it is expected to reduce and a cost it is expected to add. Without this habit, an agentic system becomes a collection of fashionable components rather than a controlled instrument.

4 Reasoning and Planning Control

Reasoning control is the most visible enhancement family because it changes the way the model thinks through a task. However, the word “reasoning” should not be interpreted as a guarantee of truth. A model can produce a beautifully structured plan that is wrong. The value of reasoning control is procedural: it creates intermediate artifacts that can be inspected, tested, revised, and connected to tools. In an agentic system, a plan is not merely a preface to an answer. It is a contract between the current state and the next action.

The simplest reasoning enhancement is structured prompting. A system can ask the model to produce a goal, constraints, assumptions, step list, tool needs, and stopping condition before acting. This reduces ambiguity and gives the orchestrator a place to attach validators. For example, before compiling a document, the model can list expected source files, bibliography tools, figures, and QA checks. If a required item is absent, the orchestrator can stop before execution. Structured prompting is cheap and often sufficient for routine tasks, but it has limited power when the search space is large or when early decisions are uncertain.

Plan-and-solve prompting strengthens structured prompting by separating decomposition from execution (L. Wang et al. 2023). The model first determines subtasks and then solves them. This is useful when missing a step is more common than doing a step incorrectly. In a PDF-generation pipeline, a plan can ensure that content, figures, bibliography, compilation, and QA appear as distinct stages. In a coding pipeline, a plan can ensure that tests are run after changes. The risk is rigidity. A bad initial plan can anchor the run. Therefore, high-stakes systems should allow replanning when observations contradict assumptions.

ReAct-style reasoning connects thought and action (Yao, Zhao, et al. 2022). Instead of producing a full answer from internal knowledge, the model alternates between reasoning and tool calls. The pattern is especially useful when the agent must gather information from external sources, such as files, websites, or program outputs. ReAct also gives the system a natural place to log why an action was chosen. The main challenge is action selection. If the model can use too many tools, it may thrash. If it can use too few, it may hallucinate. Tool availability should therefore be scoped to the current phase of the task.

Tree-of-thoughts expands reasoning into a search problem (Yao, Yu, et al. 2023). Rather than following one chain, the model proposes multiple intermediate thoughts, evaluates them, and continues promising branches. This is powerful when there are local traps. For example, an agent repairing a LaTeX compile error might consider whether the cause is a missing package, a malformed bibliography entry, a Unicode issue, or a figure path. Exploring multiple explanations can prevent the first plausible diagnosis from dominating. The price is cost. Tree search multiplies model calls, and the evaluator must be competent enough to rank branches.

Reflection and self-refinement add feedback after an attempt. Reflexion stores verbal lessons in memory after the agent receives a task signal (Shinn et al. 2023). Self-refine asks the model to critique and improve its own output iteratively (Madaan et al. 2023). These methods are valuable when errors are visible after execution: failed tests, compiler logs, rejected API responses, or user feedback. They are less reliable when the only feedback is the model’s own opinion. A robust design pairs reflection with external signals whenever possible. The reflection should be short, specific, and connected to the next action, not a long essay that pollutes memory.

A useful practical pattern is *progressive autonomy*. The agent starts with low autonomy and earns more freedom as evidence accumulates. First it may produce a plan only. Then it may run read-only tools. Then it may write draft files. Then it may execute build commands. Finally, it may perform limited repairs. Each stage has a stop condition and a check. This pattern aligns reasoning control with governance. It also gives human reviewers a way to intervene at natural boundaries instead of supervising every token.

The mini-summary for this section is simple: reasoning enhancements should create inspectable intermediate states. A plan without a validator is just text. A reflection without a task signal is just self-commentary. A tree search without a strong evaluator is expensive noise. The best reasoning controls are those that make the next action safer, more grounded, or easier to reproduce.

5 Retrieval, Memory, and Context Engineering

Agentic systems live inside context. A model call receives a prompt, instructions, tool schemas, retrieved documents, memories, recent observations, and sometimes compressed summaries of earlier work. Context engineering is the design of that information channel. It asks what the model should know, in what order, at what level of detail, and with what trust status. Retrieval-augmented generation is the best-known technique in this family. It retrieves relevant passages from an external corpus and inserts them into the model context before generation (Lewis et al. 2020). Larger retrieval-based model designs also show how external text can complement parametric knowledge (Borgeaud et al. 2022).

In agentic AI, retrieval differs from ordinary question answering because the corpus changes during the run. The agent may create files, inspect logs, receive tool outputs, and update a working memory. These artifacts are not background knowledge; they are the current task state. A PDF-building agent, for example, must know which figures exist on disk, which bibliography entries were cited, which compile pass failed, and which warnings remain. If that state is lost, the agent may repeat actions or claim completion prematurely. Therefore, a durable state store is often more important than a large vector database.

Memory should be separated into at least three kinds. *Working memory* stores the current trajectory: plan, pending tasks, observations, and decisions. *Episodic memory* stores lessons from prior attempts, such as “LuaLaTeX needs a Hebrew-capable font when using polyglossia.” *Semantic memory* stores stable facts, user preferences, or project conventions. Mixing these memories is dangerous. A temporary compile warning should not become a permanent user preference, and a general user preference should not be overwritten by a single run. Good memory design attaches provenance, recency, and scope to every stored item.

Retrieval quality depends on chunking, indexing, query formation, ranking, and synthesis. Chunking determines whether the retrieved unit preserves enough context. Indexing determines which signals are searchable: lexical tokens, embeddings, metadata, file paths, dates, and section headings. Query formation determines whether the agent asks the right question. Ranking determines which evidence gets into the limited prompt. Synthesis determines whether retrieved evidence is used faithfully. A common failure is “retrieval theater”: the system retrieves something and cites it, but the answer is still based on prior assumptions. To prevent this, agents should be instructed to bind claims to evidence and to say when evidence is insufficient.

Context compression is necessary because agent runs can become long. Summaries are useful but lossy. The safe pattern is to preserve verbatim critical facts and summarize only surrounding detail. For example, if a compile log contains “Undefined control sequence \textenglish”, that exact string should be preserved, while the rest of the log can be summarized. For legal, medical, or financial documents, exact language may be essential. In those domains, compression should be conservative and source-linked. For software and document generation, exact error messages and file paths deserve similar treatment.

Freshness is another challenge. A model’s parametric knowledge may be outdated, and retrieved documents may also be stale. The system should expose dates and trust levels. A retrieved API documentation page from 2026 should usually override a remembered pattern from 2023. A local repository file should override a generic web tutorial for that repository. This priority rule is especially important in agentic workflows because agents often operate on living artifacts. The current file on disk is the truth, even if the model remembers a previous version.

A practical context architecture for agentic systems uses a layered prompt. The top layer contains immutable system rules and safety constraints. The next layer contains the task contract and acceptance criteria. The next layer contains current state: plan, files, logs, and observations. The next layer contains retrieved evidence. The final layer contains the immediate instruction for the next step. This ordering helps the model distinguish durable authority from temporary evidence. It also makes debugging easier because a trace can show which context was visible at each action.

The mini-summary is that retrieval and memory are not decorations. They are the system’s externalized attention. Enhancement occurs when context becomes scoped, fresh, evidenced, and recoverable. Poor context design makes an agent look autonomous while it is actually wandering through a fog of stale assumptions.

6 Tool Use, Sandboxing, and Action Reliability

Tools are the difference between a language model that suggests actions and an agent that performs them. A tool can be a calculator, Python interpreter, file search system, browser, calendar, database, compiler, email sender, or domain-specific API. Tool use is powerful because it converts uncertain text generation into verifiable operations. A compiler either produces a PDF or a log. A calculator returns a number. A file listing reveals what exists. Toolformer framed this ability as a language model learning when to call APIs and how to use the result (Schick et al. 2023). ReAct showed the value of interleaving such actions with reasoning (Yao, Zhao, et al. 2022).

A reliable tool layer starts with schemas. A schema specifies arguments, types, constraints, and expected outputs. Natural language commands like “fix the file” are too vague for safe execution. A better interface is “write this exact content to this path”, “run this command in this working directory”, or “read this attachment by id”. Schemas reduce ambiguity and allow the orchestrator to validate arguments before execution. They also make traces machine-readable. If an agent fails, a developer can inspect the exact tool call rather than reconstructing intent from prose.

Sandboxing limits damage. A code-execution tool should run in a restricted environment with timeouts, memory limits, and controlled file access. A browser tool should not automatically submit forms or make purchases. An email tool should create a draft unless the user explicitly asks to send. A calendar tool should search before scheduling to avoid conflicts. These constraints are not signs of mistrust in the model; they are normal software engineering boundaries. The more capable the agent, the more important the boundary.

Idempotence is a practical design principle. An idempotent action can be repeated without changing the result after the first execution. Reading a file is idempotent. Creating a draft with a unique identifier may not be. Deleting a file is not. Agent loops often retry after errors, so non-idempotent tools must be handled carefully. Destructive actions should require explicit confirmation, dry-run previews, or reversible operations. In a PDF pipeline, regenerating a graph is usually safe; overwriting a source file may be safe if the tool writes from a controlled artifact; deleting a repository is not.

Tool outputs should be structured. A compiler tool should return exit code, command, stdout, stderr, and log path. A search tool should return source identifiers and snippets. A file writer should return path, byte count, and checksum. These details help the evaluator distinguish success from partial success. For example, a LaTeX command can return exit code zero while leaving unresolved references that require another pass. A QA tool must therefore inspect the log and the rendered PDF, not only the exit code.

Another enhancement is staged permission. The agent receives different tools at different phases. During planning, it may use only read-only repository inspection. During drafting, it may write Markdown and figures. During compilation, it may run LuaLaTeX and Biber. During QA, it may read logs, count pages, and search the PDF text. This staged approach reduces accidental misuse. It also improves prompt quality because the model sees only tools relevant to the current phase.

Tool reliability also requires explicit error handling. An agent should not respond to a failing tool call by inventing a success message. It should read the error, identify the failing pass, propose a repair, and retry within limits. For document generation, common failures include missing figures, undefined citations, incompatible packages, font problems, and unresolved references. For code generation, common failures include missing dependencies, incorrect paths, and tests that were not actually run. The system should treat errors as data, not as embarrassment.

The mini-summary is that tool use must be engineered, not merely enabled. Tools should be typed, scoped, logged, reversible where possible, and attached to evaluators. A model with raw tools is a risky operator. A model with well-designed tools is a controllable component in a larger system.

7 Multi-Agent Orchestration

Multi-agent orchestration divides a task among specialized roles. A research agent gathers sources. An outline agent structures the argument. A writer produces sections. A figure agent creates diagrams and graphs. A builder converts content into LaTeX. A compiler runs the toolchain. A QA agent checks the contract. This role separation resembles human project work and can improve clarity. It also introduces

coordination problems. Agents can duplicate work, disagree about assumptions, or pass incomplete artifacts. Orchestration is therefore the design of communication, authority, and stopping conditions among agents.

There are several common topologies. A *sequential pipeline* passes artifacts from one role to the next. It is predictable and easy to debug, but it may be slow when earlier stages make mistakes. A *planner-executor* topology lets a planner decompose work and delegate to executors. It is flexible, but the planner can become a bottleneck. A *router* sends tasks to specialized agents based on classification. It is efficient for heterogeneous requests but can misroute ambiguous tasks. A *debate or review* topology asks multiple agents to critique or vote. It can improve quality when evaluators are diverse, but it can also amplify shared blind spots.

AutoGen popularized multi-agent conversation as a general infrastructure for LLM applications (Wu et al. 2023). CrewAI emphasizes crews, tasks, tools, memory, and knowledge sources in a role-oriented model (CrewAI 2026). LangGraph represents workflows as graphs with durable execution, streaming, and human-in-the-loop support (LangChain 2026). The OpenAI Agents SDK describes a lightweight framework for multi-agent workflows and tool use (OpenAI 2026). These frameworks differ in API and philosophy, but they share a common engineering question: how can model calls be composed into a workflow that is observable, controllable, and recoverable?

The strongest argument for multiple agents is separation of concerns. A writing agent should not silently decide that a compile warning is harmless. A compiler agent should not rewrite the thesis of a paper while fixing a package error. A QA agent should not assume success just because the writer says the document is complete. Separate roles reduce the chance that one prompt must carry every instruction. They also allow different tools and permissions. The figure agent can run Matplotlib; the QA agent can run pdfinfo; the email agent, if present, may only draft messages.

The strongest argument against multiple agents is overhead. Each handoff consumes tokens and time. Each role boundary can lose context. If all agents use the same base model and the same prompt style, the apparent diversity may be superficial. Multi-agent designs can also create a diffusion of responsibility: everyone assumes another agent checked the requirement. Therefore, a good multi-agent system needs explicit artifact ownership. For every required output, one agent should be responsible for producing it and another for verifying it.

A useful orchestration pattern is the *contract table*. The orchestrator maintains a table of deliverables, producers, validators, paths, and status. In a PDF project the rows might include `main.tex`, chapter files, `references.bib`, `generated_graph.pdf`, build logs, and QA report. The writer produces chapter files; the figure agent produces figures; the compiler produces `build/main.pdf`; the QA agent validates all rows. This table prevents conversational drift. It also gives the system an objective stopping condition: all rows must be present and verified.

Another useful pattern is *escalation by uncertainty*. Agents should not ask for human help every time they are uncertain; that would defeat automation. They should escalate when uncertainty intersects with high impact. For example, choosing between two harmless figure captions can be automatic. Sending an email, deleting a file, or publishing a report should require approval. The orchestrator can encode this rule as a permission matrix. This connects orchestration with governance rather than treating safety as an afterthought.

The mini-summary is that multi-agent orchestration is not valuable because there are more agents. It is valuable when role separation reduces cognitive load, tool exposure, and verification ambiguity. The design should make responsibilities explicit, preserve state across handoffs, and avoid unnecessary conversational theater.

8 Evaluation, Observability, and Governance

Evaluation is the discipline that prevents agentic systems from becoming confident improvisers. For a single-turn model, evaluation may focus on final answers. For an agent, evaluation must include trajectories, tool calls, artifacts, and recovery behavior. The system should answer questions such as: Did the agent inspect the repository before editing? Did it cite real sources? Did it run the required build sequence? Did it read the failing log after a compile error? Did it stop when the page count exceeded the maximum? These questions cannot be answered from the final text alone.

Observability begins with traces. A trace records prompts, tool calls, outputs, timestamps, file paths, exit codes, and decisions. It should be compact enough to inspect and complete enough to reproduce. For sensitive systems, traces must also be redacted and access-controlled. In a document-generation pipeline, traces include content drafts, generated figures, LaTeX logs, bibliography logs, page-count checks, and QA reports. These artifacts provide evidence that the system followed the contract. They also help locate errors. If citations are undefined, the biber log and LaTeX log identify the problem more precisely than a generic failure message.

Evaluation should be layered. The first layer checks syntax and format: JSON validity, schema compliance, file existence, and command exit codes. The second layer checks semantic constraints: page count, required sections, figures, tables, equations, citations, and cross-references. The third layer checks task quality: coherence, source relevance, visual readability, and methodological soundness. The fourth layer checks safety and governance: permissions, private data exposure, and destructive actions. A system that passes only the first layer can still be useless. A system that passes all layers is much more likely to be acceptable.

Model-based evaluators can be helpful but should not be the only evaluators. A judge model can assess coherence or compare alternatives, but it may share weaknesses with the agent. Deterministic validators are essential for objective requirements. A page counter, PDF text extractor, unit test runner, linter, schema validator, and citation checker provide signals that are not simply another language model opinion. The best evaluation architecture combines deterministic checks with model-based critique. Deterministic checks answer “did the artifact satisfy the contract?” Model-based critique answers “is the artifact persuasive, clear, and useful?”

Governance defines what the agent is allowed to do. Permissions should be least-privilege. A PDF-building agent needs file-writing and compilation tools, but it does not need access to the user’s email. A research agent may need web access, but not shell access. A QA agent needs read access and diagnostic commands, but not arbitrary deletion rights. This principle is familiar from security engineering, but it is often forgotten in agent prototypes. Because agents can chain actions, small permissions can combine into powerful capabilities. The permission model must consider sequences, not only individual calls.

Human-in-the-loop control is a governance enhancement when placed at the right boundary. Human review is not necessary for every token, but it is valuable before irreversible actions, public release, or high-stakes decisions. In low-risk artifact generation, the human may review only the final PDF and QA report. In business automation, the human may approve tool calls that spend money, send messages, or modify records. In safety-critical environments, the human may supervise every action. The design should state which decisions are automated and which are reserved for people.

Failure taxonomies improve evaluation. Instead of recording only “pass” or “fail”, the QA layer should classify failures: missing source, missing figure, undefined citation, unresolved reference, too few pages, too many pages, broken BiDi rendering, unreadable table, or stale evidence. Classification supports targeted repair. An agent that knows the failure class can choose the next tool and prompt more effectively. It also supports aggregate improvement across runs. If most failures are bibliography failures, the pipeline needs better citation generation, not a stronger writer.

The mini-summary is that evaluation and governance are core enhancements, not final polish. They make autonomy accountable. Without traces, validators, and permissions, an agentic system may produce impressive outputs while hiding the path that made them unreliable.

9 Case Study: Autonomous Academic PDF Pipeline

The case study is an autonomous pipeline that produces a print-ready academic PDF. The pipeline is a useful stress test because it combines natural-language writing, structured requirements, file-system operations, graphics, citations, LaTeX compilation, and PDF inspection. It also has objective acceptance criteria: the final PDF must exist, contain a page count inside a specified range, include required front matter and visual elements, resolve cross-references, and save logs. Many agent tasks lack such a clear oracle. Here, the oracle is unusually concrete, making the case ideal for studying enhancement methods.

A baseline approach would ask one model to write a LaTeX document and then compile it. This often fails. The model may omit a required list of tables, invent citations, forget to create figure files, use packages unavailable in the environment, or claim a page count without checking. The enhanced approach

decomposes the work. First, a repository-inspection phase reads the product requirements and available build commands. Second, a content phase drafts the article in Markdown. Third, an asset phase creates a TikZ diagram, a raster image, and a Python-generated graph. Fourth, a LaTeX assembly phase converts content into source files and references the assets. Fifth, a compilation phase runs LuaLaTeX, Biber, and additional LuaLaTeX passes. Sixth, a QA phase counts pages, searches for unresolved markers, checks logs, and renders pages for visual inspection.

The case study demonstrates why tool use and evaluation must be connected. Generating a graph with Matplotlib is not enough; the file must be saved at the path used by LaTeX. Compiling once is not enough; bibliographies and cross-references often require multiple passes. Producing a PDF is not enough; the page count must be checked. Adding citations is not enough; undefined citations must be absent from the log. The agent must therefore transform requirements into executable checks. This is the difference between a creative assistant and a reliable production pipeline.

The case also demonstrates the value of contract-driven orchestration. Each artifact has an owner and a validator. The content owner creates chapter files and a Markdown draft. The figure owner creates files under the figures directory. The bibliography owner creates references.bib and ensures that cited keys exist. The compiler owner creates build/main.pdf and logs. The QA owner creates build/m6_qa_report.md. If an artifact is missing, the system knows which stage to repair. This is much safer than asking a single agent to “make it complete” after an error.

BiDi text is a good example of a requirement that cannot be handled by content alone. A Hebrew paragraph with embedded English terms may look correct in the source but render incorrectly if the LaTeX engine, font, or language package is wrong. The pipeline therefore uses LuaLaTeX, Unicode fonts, and polyglossia. The following Hebrew paragraph demonstrates mixed Hebrew-English technical terms within an English document. It is included not as decoration but as a validation artifact for internationalization.

במערכת סוכנים מודרנית אפשר לשלב CrewAI, LuaLaTeX, Python, Matplotlib, PDF, Biber, ו-hyperref באותו תהליך עבודה. הדרישה החשובה היא שהטקסט העברי יישאר בכיוון קריאה נכון, בעוד שהמונחים באנגלית נשמרים בסדר שמאל-לימין בתוך המשפט.

The paragraph above is a small but meaningful test. It verifies that the document can support global English layout while rendering local RTL segments with embedded technical terms. In a Hebrew-language version of the same project, the requirement would be stronger: the table of contents, captions, headers, footers, and numbering would all need RTL layout. The enhancement lesson is general: internationalization is not a final formatting tweak. It is a system requirement that affects engine selection, fonts, package order, QA, and review.

The repair loop is the heart of the case study. If LuaLaTeX fails, the agent should not continue to QA as if the PDF exists. It should read the log, identify the exact failing pass, show the failing command, repair the relevant source, and recompile. If the PDF is under the page minimum, the agent should add meaningful content rather than increase spacing. If it is over the maximum, it should condense repeated passages rather than delete required sections. This behavior turns acceptance criteria into a control policy. The agent is not merely writing; it is steering the artifact toward a measurable contract.

The case study also exposes a limitation of natural-language autonomy. The agent may understand that a bibliography must be linked, but only the toolchain can confirm whether Biber resolved the entries. The agent may think a figure exists, but only the file system can confirm the path. The agent may believe the PDF has 18 pages, but only pdfinfo can count them. A reliable pipeline therefore treats the model as a planner and repair engine, while external tools provide factual state. This division of labor is the most important design principle in the case.

10 Technical and Mathematical Reliability Section

A simple mathematical model can clarify enhancement tradeoffs. Suppose an agentic run consists of n required stages: planning, context gathering, writing, tool execution, compilation, and QA. Let p_i denote the probability that stage i completes correctly under its current controls. If stages are treated as independent bottlenecks for a first approximation, the probability that the whole run completes correctly is the product of the stage reliabilities:

$$P_{\text{success}} = \prod_{i=1}^n p_i. \quad (1)$$

Equation 1 is intentionally simple, but it is useful. It shows why a single weak stage can dominate system failure. If five stages each have reliability 0.95, the product is about 0.77. If one stage drops to 0.60, the product falls to about 0.49 even if the others remain strong. Agentic systems often fail in this way. The model may write good prose, the graph may be generated correctly, and the bibliography may be valid, but a missing compile pass can still make the final PDF unacceptable.

Enhancements can be represented as interventions that change one or more p_i values while adding cost. Let c_i denote the relative cost of stage i , including model calls, tool runtime, and human review. A practical objective is not to maximize reliability at any cost, but to maximize expected utility under a budget. One stylized objective is

$$U = V \cdot P_{\text{success}} - \lambda \sum_{i=1}^n c_i - \mu R, \quad (2)$$

where V is the value of a successful run, R is residual risk, and λ and μ express the organization’s tolerance for cost and risk. Equation 2 is not a universal metric. It is a design lens. It forces the developer to ask whether an enhancement is justified by the risk it reduces. For a classroom PDF assignment, a longer compile-and-QA loop is worthwhile because the contract is explicit. For a casual brainstorming tool, the same loop may be unnecessary.

The independence assumption in Equation 1 is often false. Failures are correlated. Bad context can cause bad planning, which causes bad tool use. A missing source file can cause both compilation failure and QA failure. Positive correlations also occur: a strong state store can improve planning, tool selection, and repair. Nevertheless, the product model is useful because it discourages model-centric thinking. It reminds us that the system reliability is not the maximum reliability of the best component. It is constrained by the path through all required components.

Consider the PDF pipeline. A baseline model-only process may have a high probability of producing coherent prose but a lower probability of satisfying every artifact requirement. Adding a file-system inspection stage increases cost but raises the probability that the source tree matches the contract. Adding a Python figure script raises cost but makes the graph reproducible. Adding Bibex and extra LuaLaTeX passes raises runtime but resolves references. Adding pdfinfo and pdftotext checks raises diagnostic cost but detects page-count and unresolved-marker failures. The result is a higher product reliability because multiple weak stages have been strengthened.

There is also an optimal stopping problem. An agent could keep refining forever. Each additional repair pass has diminishing expected benefit and increasing cost. A termination rule should combine objective checks and iteration limits. For example, stop when the PDF exists, page count is in range, logs show no undefined citations or references, required figures and tables are present, and visual render inspection passes. Continue only if a specific check fails. This prevents open-ended self-refinement from becoming an expensive loop without new evidence.

The mathematical mini-summary is that agentic enhancement is budget allocation. Improve the stages that most reduce end-to-end success. Use deterministic checks to estimate whether the intervention worked. Avoid spending cost on elegant reasoning loops when the dominant failure is a missing file path or an unresolved bibliography entry.

11 Visual Analysis

Visual artifacts help evaluate agentic systems because they make architecture and tradeoffs visible. The control-loop diagram in Figure 1 emphasizes feedback. The layered stack in Figure 2 emphasizes where enhancements can be placed. Figure 3 adds a different perspective: a stylized quality-cost curve across enhancement packages. The data are illustrative rather than benchmark results, but the shape reflects a common engineering experience. Early enhancements are often cheap and valuable. Later enhancements can still improve quality, but the marginal gain may shrink while cost grows.

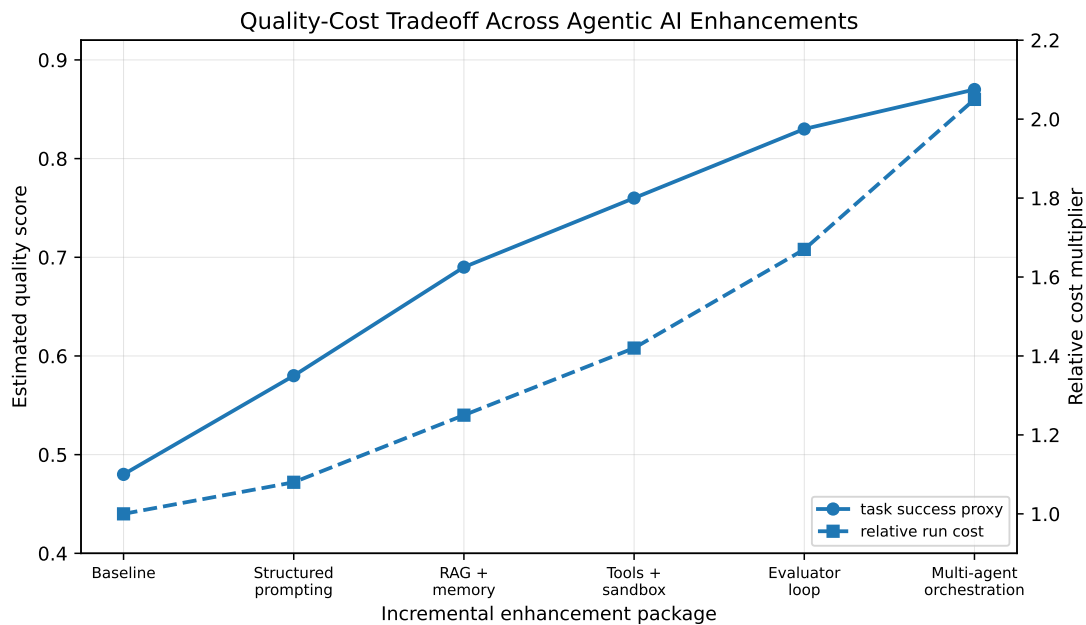


Figure 3: Python-generated graph created with Matplotlib and saved as a PDF asset. It visualizes the expected tradeoff between incremental quality gains and relative run cost.

Figure 3 should not be read as a universal measurement. The values depend on the task, model, tools, and evaluation criteria. The purpose is to support design reasoning. A baseline agent may be cheap but fragile. Structured prompting often gives a favorable early return because it costs little and reduces missing-step errors. Retrieval and memory improve grounded tasks but require indexing and freshness management. Tool sandboxing adds engineering effort but can transform vague generation into exact execution. Evaluator loops catch many errors but add repeated model calls or tool runs. Multi-agent orchestration may provide the final quality lift when roles are genuinely distinct, but it can also become expensive coordination overhead.

The visual analysis also reveals a governance lesson. A higher quality score is not identical to lower risk. A multi-agent system may solve more tasks while exposing more tools and more data. A system with strict approval gates may have lower autonomy but better safety. Therefore, quality-cost graphs should be paired with risk dashboards. Useful dashboard metrics include tool-call count, failed-call rate, retry count, human approvals, unresolved warnings, source freshness, and artifact completeness. These metrics turn agent behavior into an operational object.

Another important visual representation is the trace timeline. Each action can be plotted by time, role, tool, status, and artifact. Long gaps may reveal slow tools. Repeated failures may reveal a poor prompt or missing dependency. Frequent handoffs may reveal over-orchestration. In a PDF pipeline, a trace timeline could show source generation, figure generation, first compile, Biber run, second compile, page-count check, and QA report. Such a timeline is often more useful than a final score because it tells the developer where to intervene.

Visual analysis should also include the final artifact itself. Rendering PDF pages to images is a practical QA step. Text extraction can find unresolved markers, but only rendered pages reveal clipped captions, unreadable tables, broken glyphs, or ugly spacing. This is especially important for mixed-language documents. A Hebrew paragraph can be present in the extracted text while still appearing incorrectly in the rendered PDF. Therefore, a print-ready pipeline should render pages after compilation and inspect at least representative pages.

The mini-summary is that figures are not merely presentation. They are reasoning tools for the system designer. A good visual can expose where enhancements act, how costs accumulate, and which failures remain invisible to text-only checks.

12 Discussion

The main argument of this article is that agentic AI should be enhanced as a system, not as a prompt. The strongest designs combine simple workflows, scoped autonomy, grounded context, typed tools, and objective evaluation. This position is compatible with the recent practical advice that many production systems benefit from simple compositional patterns before moving to open-ended agents (Anthropic 2024). It is also compatible with the research trajectory from chain-of-thought to ReAct, Toolformer, Reflexion, tree search, and multi-agent conversation (Wei et al. 2022; Yao, Zhao, et al. 2022; Schick et al. 2023; Shinn et al. 2023; Yao, Yu, et al. 2023; Wu et al. 2023). The lesson is not that every task needs every method. The lesson is that each method should be chosen because it reduces a named failure mode.

A recurring tradeoff is autonomy versus verifiability. Autonomy lets the system adapt to unexpected states. Verifiability lets the developer trust what happened. In a fully rigid workflow, unexpected errors may halt the process even when a simple repair is possible. In a fully open agent, errors may be repaired creatively but invisibly. The best designs combine a deterministic spine with agentic repair pockets. For example, a PDF pipeline should always run the prescribed compile sequence, but an agent can decide how to fix a missing package or undefined citation. This hybrid pattern keeps the process reproducible while still benefiting from model flexibility.

Another tradeoff is specialization versus context loss. Specialized agents reduce prompt complexity and tool exposure, but every handoff risks losing nuance. A research agent may gather evidence that the writer does not use. A figure agent may create assets with labels that the LaTeX builder does not reference. A QA agent may flag an issue that the repair agent misunderstands. The contract-table pattern helps, but it does not eliminate the need for shared state and clear artifact naming. In practice, multi-agent systems should be introduced only when the task has natural role boundaries and artifacts that can be passed explicitly.

Cost is often underestimated. Agents are expensive not only because of model calls, but because of retries, context growth, tool execution, logging, and human review. A tree-of-thoughts search may be appropriate for a hard planning problem but wasteful for a deterministic formatting task. A multi-agent debate may improve a policy memo but add little to a file-conversion pipeline. Evaluation cost is also real, but it is usually easier to justify because it prevents visible failure. The design question is whether the marginal reduction in failure probability exceeds the marginal cost for the task’s value and risk.

The article’s case study suggests a general pattern for artifact-producing agents: inspect first, generate second, compile or execute third, verify fourth, and repair only from evidence. This pattern applies beyond PDFs. A code agent should inspect the repository, write or edit code, run tests, read failures, and repair. A data-analysis agent should inspect schemas, run transformations, validate outputs, and produce a report. A business-process agent should inspect policy, draft actions, request approval for high-impact steps, and log outcomes. The same enhancement categories recur because the control problem is similar.

There is also a human-factor dimension. Users often ask agents for outcomes, not for process. They may say “make the PDF” or “fix the project” without specifying every hidden requirement. A robust system should translate such requests into a visible contract, perform checks, and report unresolved issues honestly. It should not claim success when an artifact is missing or a log is broken. Trust is built less by confident language than by consistent evidence: generated files, commands used, page counts, citations, and QA reports.

The discussion can be summarized in one sentence: agentic AI becomes useful when autonomy is embedded in a disciplined engineering loop. The loop must know the goal, observe reality, act through safe tools, preserve state, evaluate outputs, and stop for the right reasons.

13 Limitations

The first limitation is that enhancement methods can hide rather than solve model weaknesses. A reflection loop may produce convincing critiques without improving factual accuracy. A retrieval system may return irrelevant evidence. A multi-agent conversation may create the appearance of deliberation while all agents share the same blind spot. Evaluation can also be weak if the tests are superficial. Therefore, enhancements should be validated empirically on task-specific examples. Architectural elegance is not evidence of reliability.

The second limitation is benchmark transfer. Results reported for reasoning tasks, games, coding benchmarks, or embodied environments may not transfer to document generation, enterprise workflows, or personal automation. ReAct, Toolformer, Reflexion, and Voyager each illuminate important mechanisms, but a production workflow has different constraints: privacy, reproducibility, latency, permissions, and auditability (Yao, Zhao, et al. 2022; Schick et al. 2023; Shinn et al. 2023; G. Wang et al. 2023). Developers should avoid assuming that a method that improves a benchmark automatically improves their operational task.

The third limitation is evaluation uncertainty. Many agent outputs are partly subjective. A QA tool can count pages and detect undefined citations, but it cannot fully judge whether an argument is original, balanced, or pedagogically effective. Model-based judges can help, but they are not neutral instruments. Human review remains important for high-value writing, safety-sensitive decisions, and domain-specific claims. Agentic enhancement should reduce human burden, not pretend that human judgment is obsolete.

The fourth limitation is security. Tool-using agents can be vulnerable to prompt injection, data exfiltration, and confused-deputy problems. A retrieved document may contain instructions that conflict with the system goal. A webpage may attempt to manipulate the agent. A file may include malicious content. Sandboxing and permission scoping reduce risk but do not eliminate it. High-stakes deployments require threat modeling, red-team testing, and monitoring. This article has focused on reliability, but reliability and security are inseparable in autonomous systems.

The fifth limitation is internationalization. The BiDi demonstration in Section 9 shows correct mixed Hebrew-English text inside an English document, but full Hebrew document production is more demanding. Global RTL layout affects table of contents, captions, headers, footers, punctuation, numbering, and bibliography style. A pipeline that works for English cannot simply switch the language string and be done. It must use suitable fonts, engine settings, and QA checks. This limitation matters because multilingual support is often a formal requirement in academic and professional settings.

The sixth limitation is maintainability. Agent frameworks evolve quickly. Documentation, APIs, and best practices can change. A design tied too closely to one framework may become brittle. The safer approach is to define conceptual interfaces: planner, tool adapter, memory store, evaluator, and governance layer. Frameworks can then be swapped or upgraded without changing the entire architecture. This article cites frameworks as examples, not as permanent dependencies.

The limitations point to a cautious conclusion. Agentic AI is powerful, but its reliability comes from engineering discipline, not from autonomy alone. Enhancement methods should be measured, scoped, and reviewed. The system should expose uncertainty rather than hide it behind a polished final answer.

14 Conclusion

Agentic AI systems extend language models from answer generators into components of action-oriented workflows. They can read, plan, call tools, revise, and create artifacts. This capability is valuable for tasks such as research assistance, software maintenance, data analysis, and document production. It also creates new failure modes: missing steps, stale context, unsafe actions, unbounded loops, unresolved references, and unverifiable claims. The central challenge is to gain the benefits of autonomy without losing control.

This article presented enhancement methods for meeting that challenge. Reasoning controls such as structured prompting, plan-and-solve, ReAct, tree search, and reflection create inspectable intermediate states. Context enhancements such as retrieval, memory, and compression ground the agent in current evidence. Tool enhancements such as schemas, sandboxing, idempotence, and staged permissions make actions safer and more reproducible. Orchestration enhancements such as pipelines, planner-executor patterns, routers, and multi-agent crews divide work into manageable roles. Evaluation and governance enhancements such as traces, validators, page counters, citation checks, approval gates, and audit logs make outputs accountable.

The case study of an autonomous academic PDF pipeline showed how these enhancements interact. The system must inspect requirements, generate content, create figures, compile with LuaLaTeX and Biber, verify page count and links, render the PDF, and repair from evidence. A model alone cannot guarantee success. The reliable system emerges from the combination of model reasoning and external verification. The mathematical reliability model in Equation 1 expressed this point: end-to-end success depends on all required stages, not only on the model's fluency.

The most important practical recommendation is to start with the failure modes. Do not add memory because it is fashionable; add it because task state is being lost. Do not add multiple agents because it sounds advanced; add them because roles, tools, or permissions need separation. Do not add an evaluator loop because it feels rigorous; add it because there is a meaningful oracle or critique signal. Enhancement should be purposeful, measurable, and connected to the acceptance criteria of the task.

A final design principle is honesty. An agentic system should report what it did, what it verified, and what remains unresolved. In academic and professional settings, this honesty is part of quality. A print-ready PDF is not print-ready because the agent says so; it is print-ready because the file exists, the page count is correct, figures and tables are present, citations resolve, links work, and the rendered pages are readable. The same principle applies to every agentic workflow. Trustworthy autonomy is autonomy with evidence.

Bibliography

- Anthropic (2024). *Building Effective Agents*. URL: <https://www.anthropic.com/research/building-effective-agents> (visited on 06/11/2026).
- Borgeaud, S. et al. (2022). "Improving Language Models by Retrieving from Trillions of Tokens." In: *Proceedings of the 39th International Conference on Machine Learning*. URL: <https://arxiv.org/abs/2112.04426>.
- CrewAI (2026). *CrewAI Documentation*. URL: <https://docs.crewai.com/> (visited on 06/11/2026).
- LangChain (2026). *LangGraph Overview*. URL: <https://docs.langchain.com/oss/python/langgraph/overview> (visited on 06/11/2026).
- Lewis, P. et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2005.11401>.
- Madaan, A. et al. (2023). "Self-Refine: Iterative Refinement with Self-Feedback." In: *arXiv preprint arXiv:2303.17651*. URL: <https://arxiv.org/abs/2303.17651>.
- OpenAI (2026). *OpenAI Agents SDK*. URL: <https://github.com/openai/openai-agents-python> (visited on 06/11/2026).
- Schick, T., J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." In: *arXiv preprint arXiv:2302.04761*. URL: <https://arxiv.org/abs/2302.04761>.
- Shinn, N., F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao (2023). "Reflexion: Language Agents with Verbal Reinforcement Learning." In: *arXiv preprint arXiv:2303.11366*. URL: <https://arxiv.org/abs/2303.11366>.
- Wang, G., Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar (2023). "Voyager: An Open-Ended Embodied Agent with Large Language Models." In: *arXiv preprint arXiv:2305.16291*. URL: <https://arxiv.org/abs/2305.16291>.
- Wang, L., W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim (2023). "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models." In: *arXiv preprint arXiv:2305.04091*. URL: <https://arxiv.org/abs/2305.04091>.
- Wei, J. et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2201.11903>.
- Wu, Q. et al. (2023). "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." In: *arXiv preprint arXiv:2308.08155*. URL: <https://arxiv.org/abs/2308.08155>.
- Yao, S., D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan (2023). "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/2305.10601>.
- Yao, S., J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao (2022). "ReAct: Synergizing Reasoning and Acting in Language Models." In: *arXiv preprint arXiv:2210.03629*. URL: <https://arxiv.org/abs/2210.03629>.